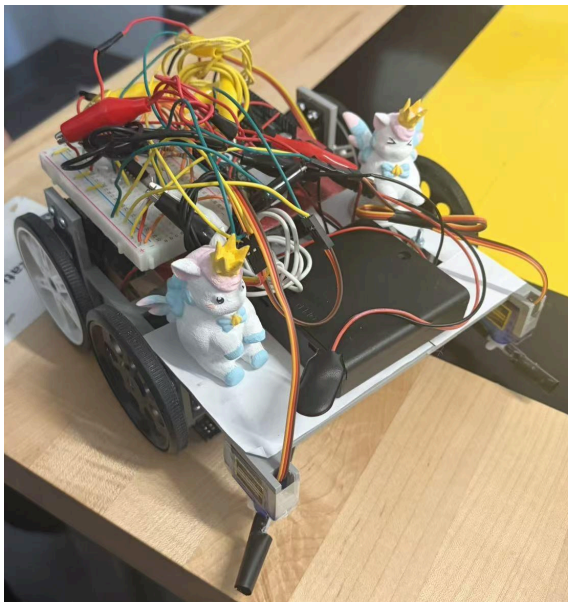
Group 19: Christine Guo (cg692), Alexa Rosano (avr42), Qing Yi Chen (qc95)

Robot Design & Overview



Our robot, F.F. ICA, consists of a custom made 3D printed PLA chassis that holds 4 servo driven wheels, 2 mini 360 degree microservos mounted in the front, and 2 QTI sensors mounted just before the front wheels. Its capabilities include forward, backward, and turning mobility, as well as line detection. The wheels and microservos are powered by a 6V battery pack while the on-board Arduino UNO is powered by a 9V battery. Connections are made on a breadboard using wires, resistors, and alligator clips. Our robot is built to be low to the ground and have powerful torque enough to not only resist being moved by opposing robots, but also push opposing robots off the board. The two microservos in front serve to brush in cubes from the sides to assist in cube collection. Ultimately, the goal of our design is to be a slow but steady immovable wall.

Design Process Reflection

For milestone 1, we based our robot overview and mechanics on a Roomba robot, which we believed to be effective at the competition's goal of collecting more blocks than our opponent. After the overview described above, we made drawings to draft the dimensions of all our components. We also added more detail to the hard-coded movement sequence which helped us prepare and structure the final competition code better and so we could translate the sequence into a flowchart for milestone 1.

With milestones 2 (movement) and 3 (color & border detection), we mainly ran into issues with debugging and not so much with building since these milestones focused on servo and sensor work. Milestone 2's debugging was due to having to adjust the delays of the turning functions in order to achieve the right turning angles so the robot could stay on the path to finish the milestone. We just kept placing the robot on the path, running the code, and adjusting the delays to slowly work toward a perfect run on the given path, allowing us to finish this milestone successfully on time. Milestone 3's problems were finding a working frequency threshold to separate blue from yellow, tuning delays again for border detection, and adjusting the code structure accordingly in order to complete the very specific series of detections and movements for the milestone. Finding the threshold was easier since it was just putting the color sensor on the board, but tuning the border delay was harder because we had to test the robot running around the board with the QTIs attached countless times. The default robot chassis couldn't mount the QTIs at the very front, but allowed us to mount slightly

behind the front, which meant by the time the QTIs detected the border, the front of the robot had already reached a little bit past the border. Because of this slight gap, we iterated through many delays to find an adequate number for the robot to detect the border in time (without being too late and accidentally falling off the board) and turn accordingly. After several runs of the color and border detection, we eventually found a working threshold and delays we needed to finish the milestone successfully on time.

For milestone 4, we had to work on our 3D-printed chassis since we were finally going to collect blocks. We wanted to 3D-print the chassis from the beginning (milestone 1) because with our goal of four-wheel drive and microservos in the front, the default chassis couldn't achieve this since it was not only small (not wide-mouthed), but also didn't have cutouts to mount four whole drive motors and the two microservos. Therefore, we chose to 3D-print so we could customly place our four drive motors, two QTI sensors, two microservos, and all our wiring at the specific positions we wanted.

Finally working on the final chassis CAD in Fusion 360, we began with some basic constraints. The bounding box allowed for the robot at the start of each round was a tight 8" x 8" shape, the wheels on the drive motors shouldn't touch each other, and the microservos needed to be far away enough so that the straws glued on the servo horns (horns attached to the microservos) didn't hit the wheels either. We initially planned for all the drive motors to be horizontal to keep center of gravity low, but the CAD showed us that this would make the robot too big for the bounding box, so we made the rear drive motors vertical to reduce space. We also had to adjust the microservo mounts because the wires were thicker than we thought and the holes to pull the wires through weren't wide enough, so we widened the holes to pull them through and connect to the Arduino + breadboard. Additionally, we shortened the height of the QTI mounts because the QTIs needed to be a certain distance from the ground and the existing mounts put the QTIs too close to the ground, so we had to change heights. With a finalized chassis, we built our physical robot and wrote a working block collection code to finish milestone 4 successfully on time. The final adjustments we made before competition were to code delays because it took many trials to perfect the timing of the final robot to react to the borders in time and not fall off the board (QTIs were placed a bit further behind the front again because the spinning straws made mounting at the front impossible).

Overall, the main problem for milestones was tuning the delays. For competition, it was countlessly tuning delays as well, fitting lots of components into a super tight bounding box, and balancing fragile structural integrity while being forced to severely cut down on chassis volume to stay within the thinning budget due to expensive 3D-printing costs.

<u>Competition Analysis</u>

On competition day, our robot performed much better than we expected throughout the tournament. During the Round Robin, our robot only lost its first match, with many large point differences, which secured us second place in our group and the opportunity to

advance to single elimination. We had not expected to make it that far at all. Surprisingly to us, most of our wins came from our robot bumping into the opposing team and either pushing them off the board or stealing their blocks. This strategy could be attributed to our large pushing force, lower center of gravity, and heavy weight. Midway through the competition, we decided to embrace this strategy and aimed our robot towards the lighter robots instead of the corner we had planned for. The same trend continued into the single-elimination rounds, where we reached the finale and earned second place after two ties.

In this final match, we had the chance to see our robot's weaknesses after its strengths had gotten us this far. Going into this competition, we were aware that our robot was slow, and if the opposing team was faster and able to avoid us, then we would have no chance at winning. Another big weakness was that our path was hard-coded. In theory, this wasn't too big a weakness. However, when programming the timings into our code, we realized the turns heavily relied on the health of our battery. This meant that further into the competition, our robot would perform worse as it could not complete its path. We originally believed the final round came with new batteries (according to the rules), but this was not enacted, and it added to our robot's unoptimized pathing and much slower movements. In the end, our robot's weight was both its biggest strength and biggest downfall. With such a small allotted time of 1 minute, we were unable to fully traverse the board to collect any remaining stray blocks. Overall, the robot's biggest strengths were its durability and pushing power, while its biggest weaknesses were its slow speed and rigid pathing; despite this, it consistently exceeded our expectations and proved highly effective at stealing and hoarding blocks throughout the competition.

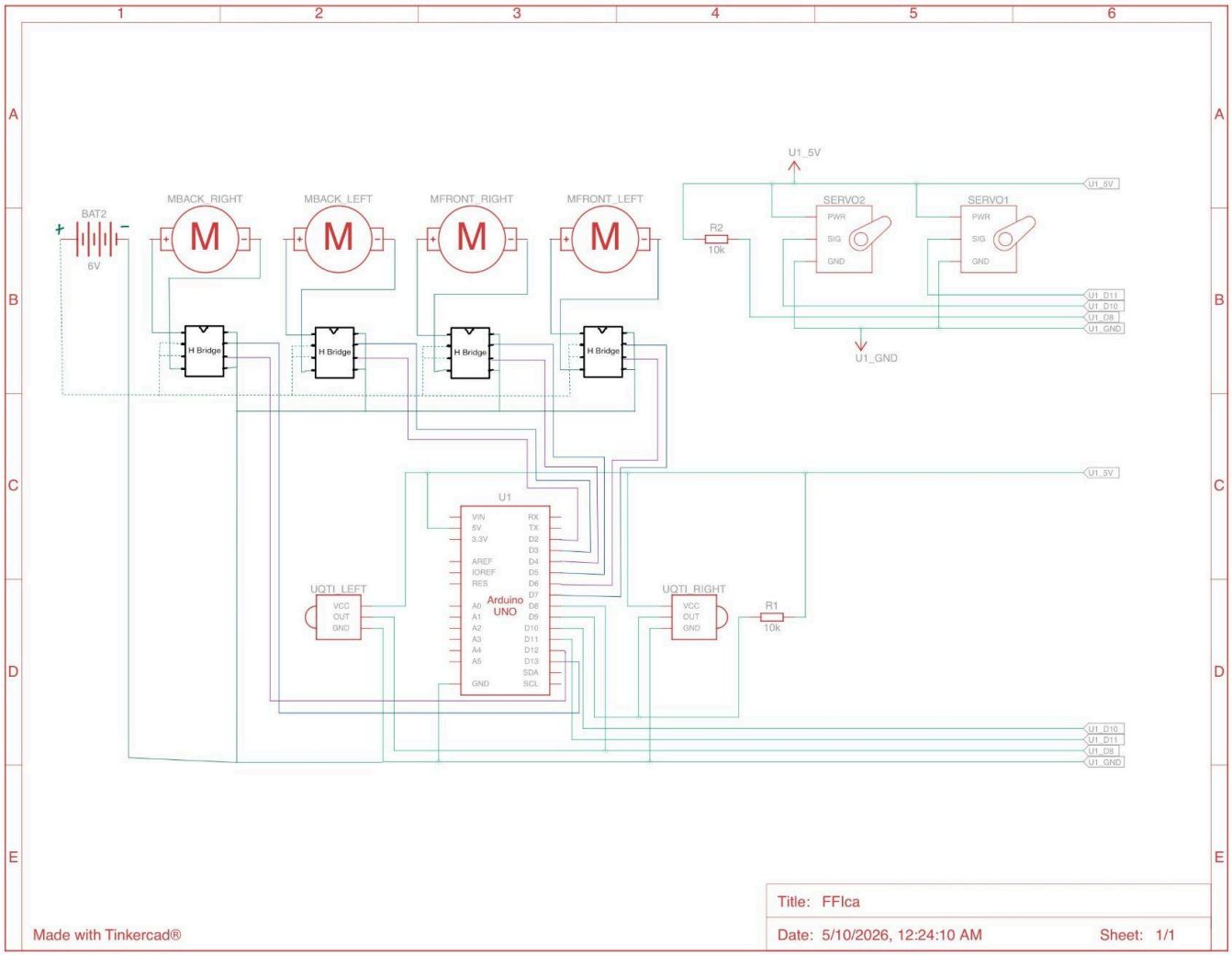
Conclusions

While our design was largely successful, one of the main changes we would likely input is testing the possibility of using larger diameter wheels. As stated above, our main weakness was the slow speed of our robot, which we decided was a fair trade in return for being immovable. However, more speed is always good in a competition like this, so we would definitely reconsider some of the speed tradeoffs we made initially if given the chance. Another change we would like to implement is the possibility of using the same brand of servos for the motors. A slight issue we ran into with our 4 wheel drive design was that some servos spun slower or faster than others, which resulted in our robot not being able to drive in a straight line. This can hopefully be better managed with 4 servos of the same brand being used to drive the wheels instead of 2 different brands. A large majority of our budget went towards 3D printing our chassis, which may not have been the best move in terms of money but served us well in terms of being able to achieve our heavyweight, low-to-the-ground build. However, we would advise upcoming students that may be doing this competition to stay away from 3D printing large pieces like our chassis simply due to how the rest of our equipment was constrained by the remaining budget. Overall, our robot was extremely successful in what we wanted it to do!

Appendix A: Bill of Materials (BOM)

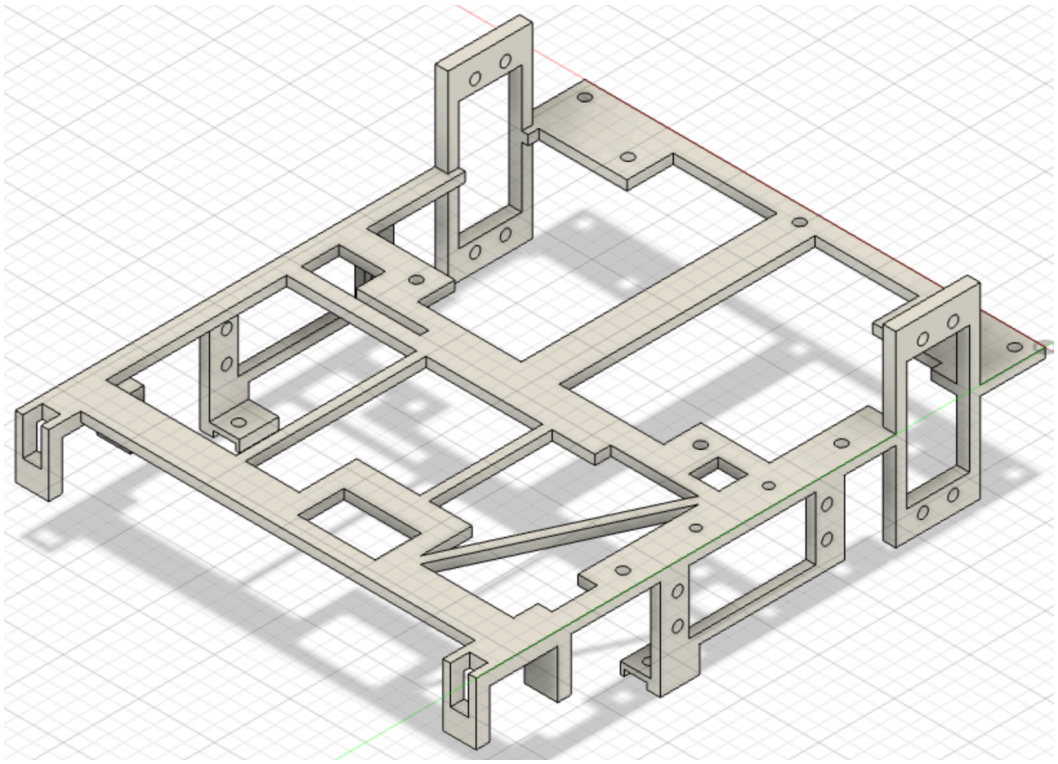
Item	Source	Unit Cost	Quantity	Total Cost
Modified Servo	Lab Store	$3	2	$6.00
Continuous Micro Servo	Lab Store	$3	1	$3.00
Wheel for Standard Servo Splines (25T, 5.8mm) - 70×8mm, White, 2-Pack	Pololu	$7.75	1	$7.75
Silicone Tire Pair for 60×8mm/70×8 mm Pololu Wheels	Pololu	$3.75	1	$3.75
3D-Printed Chassis (PLA)	RPL	$1 + $0.40/g	43.19 grams	$18.28
Straw	Recycled	$0.15 Estimated	1	$0.15
Cardboard	Recycled	$1 / Small Sheet	1	$1
			Total	$39.93
			Budget Remaining	$0.07

Appendix B: Circuit Diagram

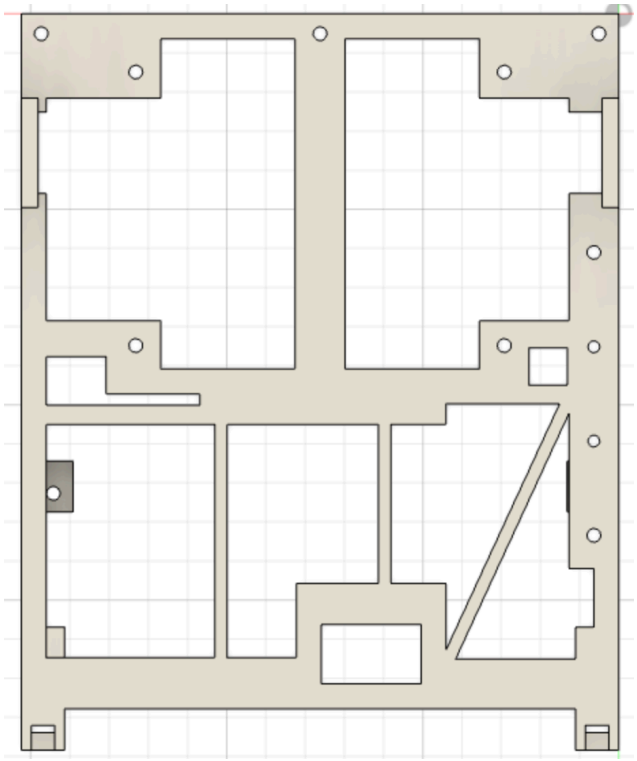


Appendix C: CAD Files and Drawings

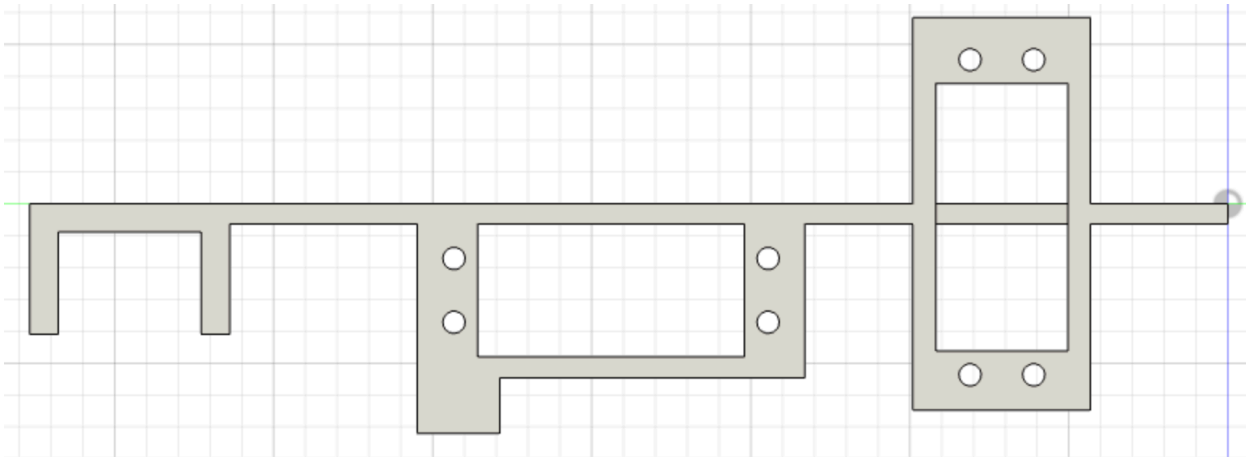
3D-Printed PLA Chassis - Orthographic View:



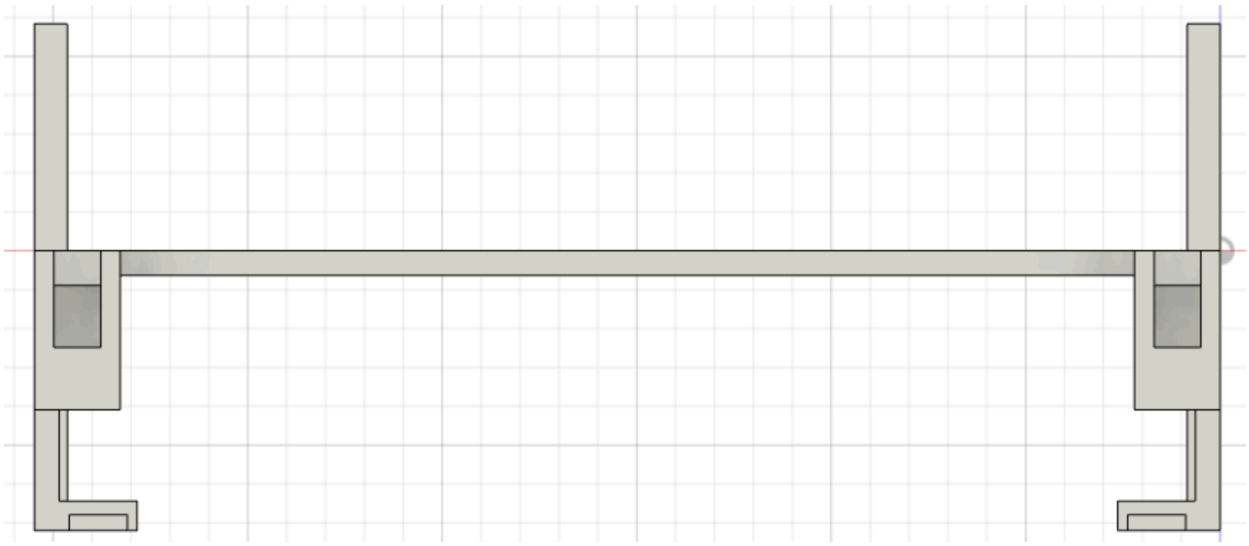
3D-Printed PLA Chassis - Top View:



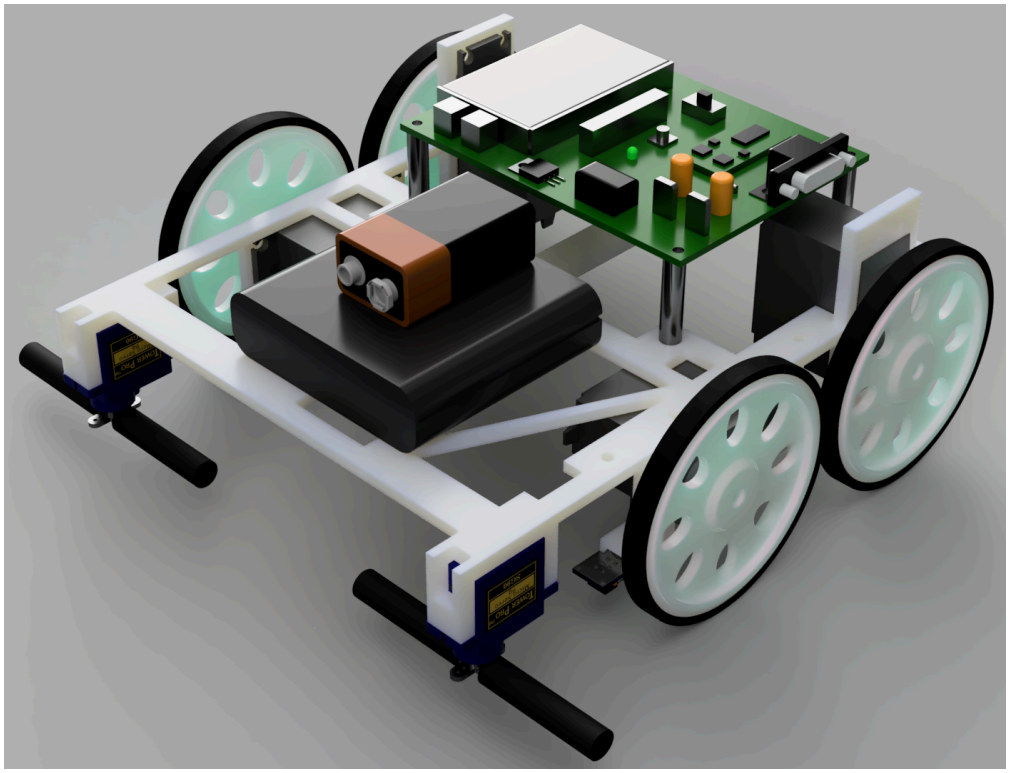
3D-Printed PLA Chassis - Side View:



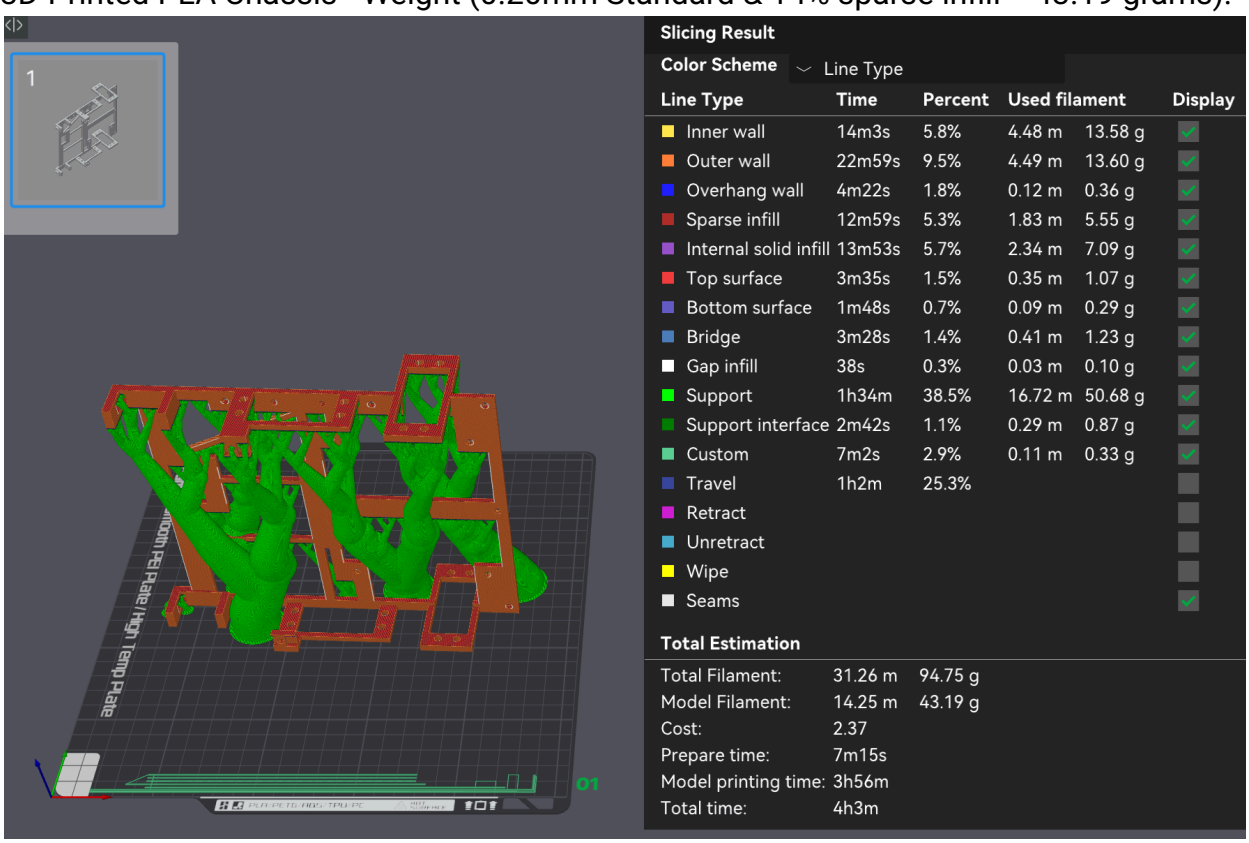
3D-Printed PLA Chassis - Front View:



Full Robot Render - Orthographic View:



3D-Printed PLA Chassis - Weight (0.20mm Standard & 14% sparse infill = 43.19 grams):



Slicing Result

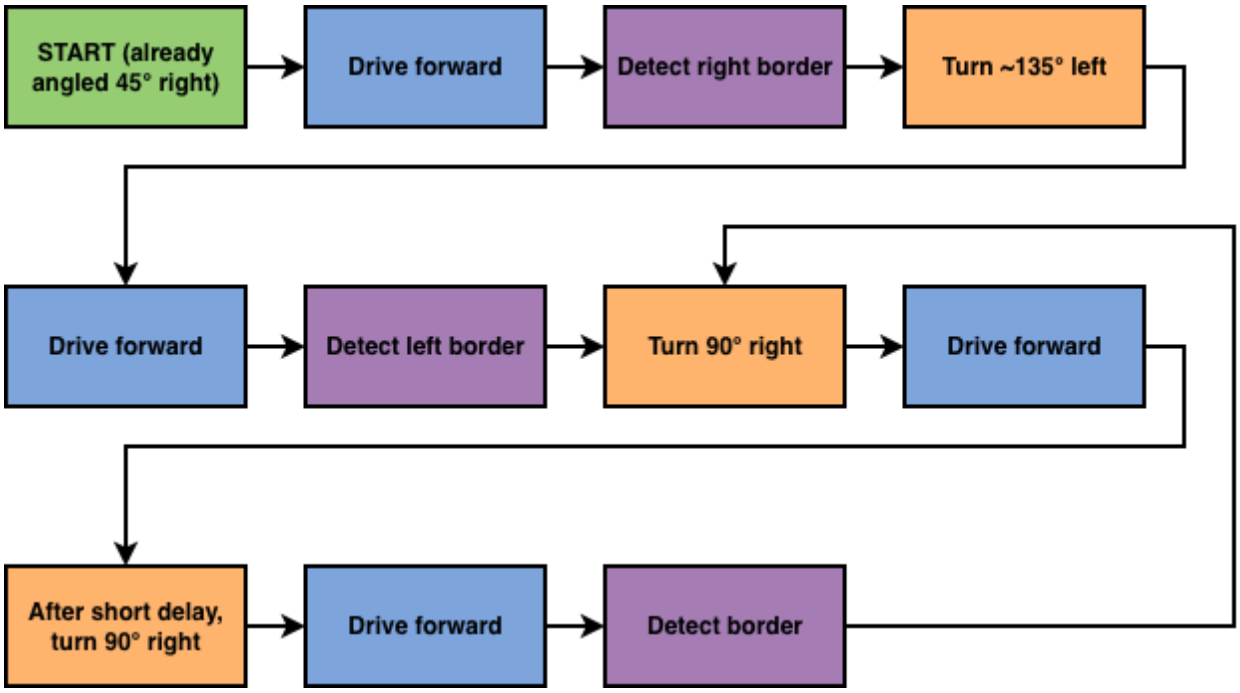
Color Scheme Line Type

Line Type	Time	Percent	Used filament		Display
Inner wall	14m3s	5.8%	4.48 m	13.58 g	✓
Outer wall	22m59s	9.5%	4.49 m	13.60 g	✓
Overhang wall	4m22s	1.8%	0.12 m	0.36 g	✓
Sparse infill	12m59s	5.3%	1.83 m	5.55 g	✓
Internal solid infill	13m53s	5.7%	2.34 m	7.09 g	✓
Top surface	3m35s	1.5%	0.35 m	1.07 g	✓
Bottom surface	1m48s	0.7%	0.09 m	0.29 g	✓
Bridge	3m28s	1.4%	0.41 m	1.23 g	✓
Gap infill	38s	0.3%	0.03 m	0.10 g	✓
Support	1h34m	38.5%	16.72 m	50.68 g	✓
Support interface	2m42s	1.1%	0.29 m	0.87 g	✓
Custom	7m2s	2.9%	0.11 m	0.33 g	✓
Travel	1h2m	25.3%			
Retract					
Unretract					
Wipe					
Seams					✓

Total Estimation

Total Filament:	31.26 m	94.75 g
Model Filament:	14.25 m	43.19 g
Cost:	2.37	
Prepare time:	7m15s	
Model printing time:	3h56m	
Total time:	4h3m	

Appendix D: Flowchart



Appendix E: Code

```
// Micro continuous servo
#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <Arduino.h>

#define SERVO_BIT (1 << PB3)   // D11
#define SERVO_BIT2 (1 << PB2)  // D10

#define SERVO_PERIOD_TICKS 1000
#define SERVO_SPIN_TICKS   125     // 85 for one way, 65 for another
#define SERVO_SPIN_TICKS2  20

volatile uint16_t servo_counter = 0;

ISR(TIMER2_COMPA_vect) {
   servo_counter++;

   if (servo_counter >= SERVO_PERIOD_TICKS) {
```

```
        servo_counter = 0;
        PORTB |= SERVO_BIT;       // start pulse HIGH
        PORTB |= SERVO_BIT2;
    }

    if (servo_counter == SERVO_SPIN_TICKS) {
        PORTB &= ~SERVO_BIT;      // end pulse LOW
    }

    if (servo_counter == SERVO_SPIN_TICKS2) {
        PORTB &= ~SERVO_BIT2;
    }
}

void init_servos(void) {
    // Pin 11 & 10
    DDRB |= SERVO_BIT;
    DDRB |= SERVO_BIT2;

    // Start LOW
    PORTB &= ~SERVO_BIT;
    PORTB &= ~SERVO_BIT2;

    // Timer2 CTC mode
    TCCR2A = (1 << WGM21);
    TCCR2B = 0x00;

    // 16 MHz / 8 = 2 MHz
    // 2 MHz = 0.5 us per timer count
    // OCR2A = 39 gives 40 counts = 20 us interrupt
    OCR2A = 39;

    // Enable Timer2 compare match A interrupt
    TIMSK2 |= (1 << OCIE2A);

    // Start Timer2 with prescaler = 8
    TCCR2B = (1 << CS21);

    sei();
}

void setup() {
DDRB |= (1 << PB2); // Set PB1 (pin 10) as output
```

```
TCCR1A = 0;
TCCR1B = 0;

// Fast PWM mode 14 (ICR1 as TOP)
TCCR1A |= (1 << COM1A1) | (1 << WGM11);
TCCR1B |= (1 << WGM12) | (1 << WGM13);

// Prescaler = 8
TCCR1B |= (1 << CS11);

// 20 ms period (50 Hz)
ICR1 = 40000;

// Clockwise rotation (~1600–1700 µs)
OCR1A = 2000; // start moderate speed
}

// driving functions
void forward() {
  // front left
  PORTD |= 0b10000000; // Set pin 7 HIGH
  PORTD &= 0b10111111; // Set pin 6 LOW

  // front right
  PORTD |= 0b00100000; // Set pin 5 HIGH
  PORTD &= 0b11101111; // Set pin 4 LOW

  // back left
  PORTD |= 0b00001000; // Set pin 3 HIGH
  PORTD &= 0b11111011; // Set pin 2 LOW

  // back right
  PORTB |= 0b00100000; // Set pin 13 HIGH
  PORTB &= 0b11101111; // Set pin 12 LOW

}

void backward() {
  // front right
  PORTD |= 0b00010000; // Set pin 4 HIGH
  PORTD &= 0b11011111; // Set pin 5 LOW

  // front left
  PORTD |= 0b01000000; // Set pin 6 HIGH
```

```
  PORTD &= 0b01111111; // Set pin 7 LOW

  // back left
  PORTD |= 0b00000100; // Set pin 2 HIGH
  PORTD &= 0b11110111; // Set pin 3 LOW

  // back right
  PORTB |= 0b00010000; // Set pin 12 HIGH
  PORTB &= 0b11011111; // Set pin 13 LOW
}

void turnRight() {
  // front right
  PORTD |= 0b00010000; // Set pin 4 HIGH
  PORTD &= 0b11011111; // Set pin 5 LOW

  // back left
  PORTD |= 0b00001000; // Set pin 3 HIGH
  PORTD &= 0b11111011; // Set pin 2 LOW

  // front left
  PORTD |= 0b10000000; // Set pin 7 HIGH
  PORTD &= 0b10111111; // Set pin 6 LOW

  // back right
  PORTB |= 0b00010000; // Set pin 12 HIGH
  PORTB &= 0b11011111; // Set pin 13 LOW
}

void turnLeft() { // NEED TO FIX
  // front left
  PORTD |= 0b01000000; // Set pin 6 HIGH
  PORTD &= 0b01111111; // Set pin 7 LOW

  // back left
  PORTD |= 0b00000100; // Set pin 2 HIGH
  PORTD &= 0b11110111; // Set pin 3 LOW

  // front right
  PORTD &= 0b11101111; // Set pin 4 LOW
  PORTD |= 0b00100000; // Set pin 5 HIGH

  // back right
  PORTB |= 0b00100000; // Set pin 13 HIGH
```

```
  PORTB &= 0b11101111; // Set pin 12 LOW
}

void stop(){ // Arduino keeps playing last command even when exit loop
   PORTD = 0b00000000; // shuts off all of Port D for the motors
   PORTB &= 0b00001111; // shuts off pins 12 & 13 for motor
}

int main(void){
  init();
  init_servos();

  // I/O
  DDRB |= 0b00110000; // 13,12 for wheels; 8 & 9 input for QTI
  DDRD = 0b11111100; // All port D output for wheel servos

  while(1){
    forward();
    _delay_ms(200);

    if (((PINB & 0b00000010) > 0) && ((PINB & 0b00000001)  >0)) { // both QTI lights up
      backward();
      _delay_ms(850);
      turnRight();
      _delay_ms(1000); // turn right 90 degrees
      forward();
      _delay_ms(1250); // cross color border`
      turnRight();
      _delay_ms(1100); // turn right 90 degrees
    }

    if ((PINB & 0b00000010)>0){ // right side QTI lights up
      turnLeft();
      _delay_ms(1700); // turn 135 ish degrees
    }

    if ((PINB & 0b00000001) > 0){
      turnRight(); // corrective measure in case left border detect
      _delay_ms(1200);
    }
  }
  return 0;
}
```